

Actividad 3-No presiones Esc... todavía

PROGRAMACION WAP

ALEJANDRO GARCÍA MORALES

1. Resumen Ejecutivo y Objetivos

En este informe se documenta la implementación de un programa de telemetría local basado en *event hooks* y *callbacks*. El propósito de esta práctica es comprender a nivel técnico cómo operan las técnicas de captura de entrada de información y reconocer sus implicaciones dentro de la ciberseguridad, tanto para la monitorización legítima como para el análisis de software malicioso.

Objetivos Específicos:

- Modificar el código base para incorporar persistencia de registros en un archivo de texto.
- Implementar una asociación temporal (marcas de fecha y hora) para cada evento registrado.
- Comprobar y explicar el funcionamiento del programa mediante pruebas en un entorno controlado y análisis del código desarrollado.

2. Tecnologías y Librerías Utilizadas

Para el desarrollo del script UPSLPWare.py se utilizó el lenguaje Python, empleando las siguientes librerías principales:

- `pynput.keyboard`: Módulo fundamental para el control y monitoreo de los dispositivos de entrada. Permite inyectar *hooks* en el sistema operativo para interceptar las pulsaciones antes de que sean procesadas por otras aplicaciones.
- `datetime`: Librería estándar utilizada para generar las marcas temporales (*timestamps*) con precisión de milisegundos, garantizando la correcta asociación temporal de cada evento.

3. Análisis del Código Fuente

El diseño del software se basa en la programación orientada a eventos, delegando la ejecución a dos funciones *callback* principales:

3.1. Función de Intercepción (`on_press`)

Esta función es invocada automáticamente por el sistema cada vez que se detecta una pulsación. Su flujo lógico es el siguiente:

1. **Asociación Temporal:** Se genera una cadena de texto con la fecha y hora exacta de la ejecución mediante `datetime.now().strftime("%Y-%m-%d %H:%M:%S.%f")[:-3]`.
2. **Clasificación de Entrada:** Se emplea un bloque `try-except` para determinar la naturaleza de la tecla. Si es un carácter estándar (alfanumérico), se extrae su valor (`key.char`). Si es una tecla especial (como *Shift*, *Enter* o *Tab*), se captura el error de atributo y se registra el nombre propio de la tecla.

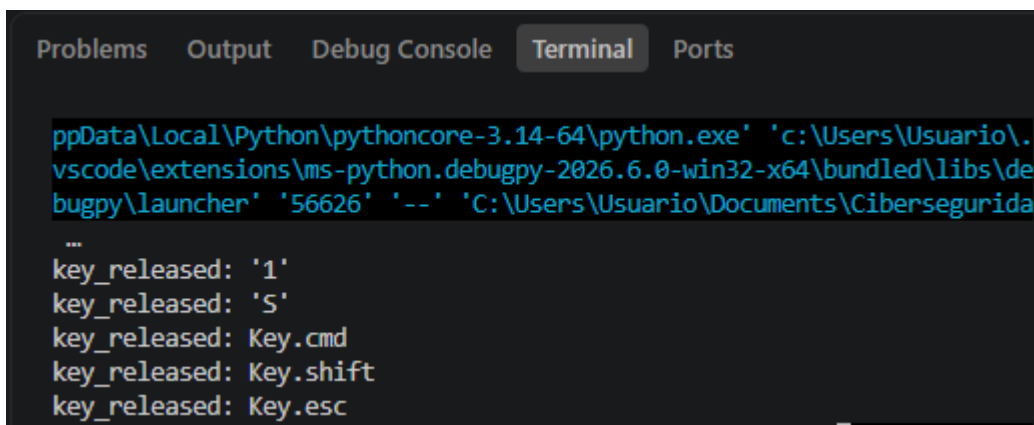
3. **Persistencia Local:** Se formatea una variable registro y se utiliza el manejador de contexto with open("Caracteres.txt", "a"). El modo a (*append*) es esencial, ya que permite añadir nuevos registros al final del archivo sin sobrescribir la información capturada previamente.

3.2. Condición de Salida (on_release)

Para mantener el control sobre la ejecución del script, la función on_release evalúa la tecla liberada. Si la tecla corresponde a Key.esc, la función retorna False, lo que rompe el ciclo del *Listener* y finaliza el programa de manera segura.

4. Evidencias de Ejecución y Pruebas

A continuación, se demuestra el funcionamiento del programa tras su ejecución en el entorno local.



```
ppData\Local\Python\pythoncore-3.14-64\python.exe' 'c:\Users\Usuario\
vscode\extensions\ms-python.debugpy-2026.6.0-win32-x64\bundled\libs\de
bugpy\launcher' '56626' '--' 'C:\Users\Usuario\Documents\Cibersegurida
...
key_released: '1'
key_released: 'S'
key_released: Key.cmd
key_released: Key.shift
key_released: Key.esc
```

Figura 1. Ejecución del script UPSLPWare.py en el entorno local.

Una vez iniciada la escucha, se procedió a teclear información de prueba para verificar la correcta interceptación, formateo y escritura en el archivo de destino.



Caracteres.txt: Bloc de notas

Archivo	Edición	Formato	Ver	Ayuda
2026-08-21	17:20:31.488	h	evento_1	
2026-08-21	17:20:32.016	o	evento_2	
2026-08-21	17:20:32.181	l	evento_3	
2026-08-21	17:20:32.293	a	evento_4	
2026-08-21	17:20:32.714	e	evento_5	
2026-08-21	17:20:32.848	s	evento_6	
2026-08-21	17:20:32.970	t	evento_7	
2026-08-21	17:20:33.037	a	evento_8	
2026-08-21	17:20:33.333	e	evento_9	
2026-08-21	17:20:33.477	s	evento_10	
2026-08-21	17:20:34.155	l	evento_11	
2026-08-21	17:20:34.267	a	evento_12	
2026-08-21	17:20:34.411	p	evento_13	
2026-08-21	17:20:34.683	r	evento_14	
2026-08-21	17:20:34.805	u	evento_15	
2026-08-21	17:20:34.885	e	evento_16	
2026-08-21	17:20:35.029	b	evento_17	
2026-08-21	17:20:35.163	a	evento_18	
2026-08-21	17:20:35.691	l	evento_19	
2026-08-21	17:20:37.393	Key.cmd	evento_20	
2026-08-21	17:20:37.449	Key.shift	evento_21	
2026-08-21	17:20:37.591	S	evento_22	
2026-08-21	17:20:44.450	Key.esc	evento_23	

Figura 2. Archivo Caracteres.txt mostrando la persistencia de los datos, el identificador de evento secuencial y la asociación temporal precisa.

5. Conclusiones y Aplicación en Ciberseguridad

A través de la implementación de UPSLPWare.py, se evidenció la facilidad con la que un lenguaje de alto nivel puede interactuar con las APIs del sistema operativo para interceptar periféricos.

Al lograr registrar la entrada del usuario de manera persistente y asociarla con marcas temporales precisas se puede entender un poco cómo un atacante puede estructurar, capturar y ocultar el flujo de datos locales.